

FEDERATION CANON

2026-03-18

Dexter Hadley, MD/PhD

Hadley Lab CANONIC

Abstract

A federation is a governed ORG with its own repo, apex, apex-signed manifest, and ledger scope
CANONIC is the permanent root, never a federation.

hadleylab.org Governed Research. Every claim cited.

Contents

1 Promotion	2
1.1 Graduation ritual (declared now; automated in Phase 4)	2
2 Credentials	2
2.1 Credential shape	2
2.2 Kinds	2
2.3 Cross-federation trust	2
2.4 Feature gate contract	3
3 Constraints	3
4 Scope note	3

- **FEDERATION_IS_ORG** every federation is one GOV repo (canonic-<name>), one apex domain, one federation_pubkey, one governance steward, one ledger scope.
- **ROOT_IS_NOT_FED** canonic-canonic is the permanent kernel root (role: ROOT in CANONIC.git). The root never appears in the federation registry.
- **REGISTRY_IS_TRUTH** canonic-canonic/FEDERATIONS/<name>/CANON.md is the single source of truth for every federation-identity field. Root CANON headers (federation_endpoint, federation_pubkey,) are a projection that MUST match the registry.
- **SCOPE_IS_NOT_FED** scopes inside an incubator federation (TALKS/, APPS/, SERVICES/*-plugin) stay scopes until they meet promotion criteria; they are NOT federations by virtue of having a domain. ## Tuple

Every entry in canonic-canonic/FEDERATIONS/<name>/CANON.md declares the following fields in YAML frontmatter:

```
# Identity
name: <slug>
repo: canonic-<name>
role: federation
bootstrap_model: incubator | de_novo
status: active | retired
inherits: canonic-canonic | canonic-

# Apex + federation protocol
apex_domain: <primary.tld>
federation_endpoint: https://galaxy.<apex>
federation_pubkey: <Ed25519 base64url>
federation_manifest_path: /.well-known/magic-manifest

# Governance
governance_model: multi_user_platform | single
governor: <user-slug>
governor_general: <user-slug>

# Cloudflare
cf_pages_project: <project-name>
cf_zone_ids: [<zone1>, <zone2>]

# Cashout (per SERVICES/COIN ğ Cashout Contract)
```

```

stripe_account_id:      <stripe_acct_id | null> # Stripe account, magic://manifest at new apex is null, route cashout yet
stripe_delegates_to:    <federation-name | null> # Federation/GRADUATED students at route here
coin_scope:             <string>
coin_scopes_delegated_here: [<scope>, ...]
                           # root+scope+scope_graduated in the source incubator

# Credentials (see § Credentials)
issues_credentials:      [tier, rank, coi_firewall, ...] # FEDERATION/GRADUATED in the source incubator
trusts_credentials_from: [<fed-name>, ...]                # whose signatures this federation honors

# Incubator-only (present when bootstrap_model=incubator)
incubator_hosted_scopes: [<scope1>, <scope2>, ...]        # scopes living inside this federation today
graduation_candidates:   [<scope1>, ...]                  # scopes actively watched for promotion

```

Any field declared in the tuple schema but not applicable to a particular federation is set explicitly to null or [] never omitted.

1. Promotion

A scope living inside an incubator federation promotes to its own federation when **any one** of these thresholds is met:

1. **Independent cashout** own Stripe account, own 1099 entity, own LLC
2. **Independent legal/compliance posture** different jurisdiction or liability stance
3. **Independent governance stewardship** governor not aligned with incubators
4. **Independent release cadence** shipping decoupled from the incubators timeline

1.1 Graduation ritual (declared now; automated in Phase 4)

1. Create `canonic-<name>` repo on GitHub; register in `CANONIC.git` manifest (role=GOV)
2. Move the scopes content tree from incubator new federation
3. Author `FEDERATIONS/<name>/CANON.md` with the full tuple
4. Regenerate shared artifacts (CORS, HOST-NAMES, cashout routing, magic peer manifests)

5. Deploy: CF zone, Pages binding, Stripe sub-account, magic://manifest at new apex
6. Ledger # FEDERATION/GRADUATED students at CANONIC# root+scope+scope_graduated in the source incubator
7. The incubators former scope path becomes a magic:// reference (no dual-write)

2. Credentials

A federation issues credentials signed with its `federation_pubkey`; any service that can resolve the issuer in the registry can verify them.

2.1 Credential shape

```

credential:
  issuer:      <federation-name>      # matches
  subject:     <user_id>               # platform
  claim:       <structured-json>       # { tier:
                                     # { board:
                                     # { board:

  valid_from:  <iso8601>
  valid_until: <iso8601 | null>        # null :
  signature:   <ed25519-base64url>    # over :

```

2.2 Kinds

- **Tier** self-declared + spot-verified (student, resident, attending). Incubator issues.
- **Rank** earned via COIN activity (listener selector curator archivist elder). Incubator issues, computed from `coin_balances`.
- **External board** federation wraps an external credential (ABOPM cert, ClinGen VCEP, CPIC contributor, state medical license) with its own signature attesting it verified the upstream.
- **Platform premium** purchased via SHOP, signed by the issuing federation.

2.3 Cross-federation trust

Default: every federation trusts its own credentials. Cross-federation trust is declared explic-

itly via `trusts_credentials_from`: in the tuple. Generators read these fields to build cross-service ACLs.

2.4 Feature gate contract

Every service that gates a feature on user state MUST declare the required credential in its `CANON.md` § Feature Gates. A shared worker helper `checkCredentials(request, required)` resolves the callers bundle (from session + D1 + federation signature verification) and returns `{ ok, reasons }`. No service re-implements gate logic.

Phase 1+2 declares the primitive; issuer + verifier code is Phase 3b (see `ROADMAP.md`).

4. Scope note

This service governs **federation identity + tuple + promotion**. The LEDGER-internal file at `../LEDGER/FEDERATION.md` governs a different concern (peer-witnessing topology at the ledger layer). Both say federation but they compose a federation tuple declared here is what a LEDGER peer witnesses.

FEDERATION / CANON / SERVICES

3. Constraints

- MUST: CANONIC is the permanent root never a federation
- MUST: Every federation has its own git repo (`canonic-<name>`), own apex domain, own governance stewardship, own ledger scope
- MUST: Federation identity fields live in `FEDERATIONS/<name>/CANON.md` source of truth for generators; never duplicate federation fields elsewhere
- MUST: Each federation is either `bootstrap_model=incubator` or `bootstrap_model=de_novo` or a graduated descendant
- MUST: Scope graduation from an incubator follows the ritual in § Promotion all 7 steps ledgered, no dual-writes, magic:// reference left behind in the source
- MUST: Federation registration ledgers `FEDERATION_REGISTERED`; graduation ledgers `FEDERATION_GRADUATED` + `SCOPE_GRADUATED`
- MUST: Root `CANON.md` headers (`federation_endpoint`, `federation_pubkey`, ...) MUST match the registry entry out-of-sync is a build-verify violation (Phase 3)
- MUST: Every service that gates a feature on user state declares the required credential in its `CANON.md` § Feature Gates
- MUST NOT: Scopes inside an incubator (`TALKS/*`, `SERVICES/*-plugin`, `APPS/*`) appear in `FEDERATIONS/` until they meet promotion criteria
- MUST NOT: CANONIC appear in `FEDERATIONS/` declared in `CANONIC.git` with `role=ROOT`
- MUST NOT: A service re-implement credential gate logic; use `checkCredentials()`